

A Preliminary Conceptual Framework for Adaptive Geopolitical Risk Analysis Using Realized and Nonparametric Betas: A Seismograph for Shocks and a Compass for Navigation

By Vincent James Hooper,

SPJ Global Working Paper September 2025

Dubai, UAE

Executive Summary

Geopolitical shocks have become the defining feature of contemporary financial markets, displacing the assumption that politics is an occasional disruptor. From Brexit to the U.S.–China trade war, Russia’s invasion of Ukraine, and recurring tensions in the Middle East, global capital markets now function under conditions of political uncertainty as the baseline, not the exception. This paper develops a framework that equips investors, policymakers, and regulators with tools better suited for this environment, moving beyond static models toward adaptive approaches. At the heart of the analysis are two complementary measures of risk sensitivity — realized betas and nonparametric betas — conceptualized as a seismograph and a compass.

Realized betas, derived from high-frequency covariance and variance, operate as a financial seismograph, detecting sudden shifts in how assets respond to benchmarks during geopolitical events. They provide near real-time alerts of volatility spikes, asymmetric contagion, and liquidity stresses. Illustrative cases include the pound’s dislocation after the Brexit referendum, semiconductor exposure during the U.S.–China trade conflict, and the sharp re-pricing of European energy equities following the Ukraine invasion. These examples reveal that beta itself is volatile, requiring monitoring as a dynamic indicator of systemic stress.

Nonparametric betas, by contrast, serve as a compass for forward-looking scenario analysis. By avoiding rigid linear assumptions, they capture state-dependent, nonlinear, and regime-specific relationships between assets and benchmarks. This flexibility exposes how sensitivities vary across political regimes: for instance, energy equities display modest betas in normal times but surge during sanctions or conflict. The framework identifies sectors with positive geopolitical betas — defense, gold, energy, cybersecurity — and those with negative exposures such as airlines, tourism, and consumer goods. This mapping

enables adaptive portfolio design and stress testing, aligning financial risk management with the realities of political disruption.

The analysis underscores regional asymmetries. In MENA, energy and currency betas spike during supply disruptions; in Asia-Pacific, technology and semiconductor betas reflect U.S.–China rivalry; in Europe, energy and equity exposures remain highly state-dependent; and in the U.S., safe-haven dynamics amplify defense-sector betas. Sentiment and behavioral factors magnify these effects, as herding and flight-to-safety flows create nonlinear contagion.

Technological advances reinforce this toolkit. Nonparametric methods align with machine learning techniques such as kernel regressions, local polynomials, and random forests, enabling adaptive and data-driven estimation of political risk exposures. Integration of these methods with liquidity monitoring reveals how shocks transmit through microstructure channels, widening spreads, evaporating depth, and exacerbating volatility. Commodities and FX markets, particularly oil, wheat, and safe-haven currencies, demonstrate nonlinear beta dynamics that encode a geopolitical risk premium into asset pricing.

To illustrate the framework in practice, the paper presents a Python-based demo system that operationalizes the seismograph and compass concepts. The Beta Seismograph class calculates rolling realized betas, issuing alerts when relationships spike or collapse, while the Beta Compass employs random forest models to generate nonlinear scenario surfaces. These are integrated into a Geopolitical Risk Analyzer that simulates crises, monitors liquidity, and performs stress testing across different regimes. The demonstration shows how synthetic market and geopolitical data can trigger real-time beta shocks, guide scenario analysis, and visualize the adaptive risk landscape, turning theory into actionable risk management tools.

The policy implications are significant. Central banks can use realized betas as near real-time stress indicators; sovereign wealth funds and pensions can deploy nonparametric betas for hedging geopolitical tail risks; and multilateral institutions can map contagion channels across borders to improve crisis preparedness. Regulators gain a tool for monitoring systemic instability, circuit breaker needs, and liquidity shocks.

Ultimately, the paper argues that political risk is financial risk. Treating geopolitical shocks as exogenous anomalies is no longer viable. By employing the seismograph-and-compass framework, market participants gain both immediate alarms and strategic navigation tools, positioning themselves to manage the ‘entropy’ and complexity of a multipolar financial order.

Introduction

Geopolitical shocks are no longer outliers — they are the rule of global finance. From Brexit to the U.S.–China trade war, from Russia’s invasion of Ukraine to tensions in the Middle East, politics is now the dominant driver of volatility. Yet the financial tools most investors rely upon were built for calmer, more linear times. They are ill-equipped to deal with the sudden spikes, nonlinear contagion, and asymmetric responses that characterize today’s geopolitics.

To navigate this new terrain, investors and policymakers need a fresh toolkit. Realized betas and nonparametric betas offer precisely that. Together, they move risk analysis away from rigid, one-size-fits-all assumptions and toward adaptive frameworks that reflect the messy reality of political risk.

Parametric, Realized, and Nonparametric Betas: A Comparative Lens

At the core of portfolio theory sits the classic beta, derived from the Capital Asset Pricing Model (CAPM). But its assumptions — stability, linearity, Gaussian distributions — collapse under the weight of geopolitical shocks.

Feature	Parametric Beta (CAPM)	Realized Beta (High-Frequency)	Nonparametric Beta (Adaptive/State-Dependent)
Assumptions	Linear, constant relationship	None (uses realized covariance/variance)	Minimal (lets data reveal nonlinear patterns)
Time Horizon	Long-term average	Short-term, high-frequency (daily/intraday)	Flexible across regimes
Crisis Behavior	Fails (misses spikes, asymmetries)	Captures sudden shifts during event windows	Models nonlinear, asymmetric sensitivities
Best Use Case	Benchmarking in stable times	Real-time event measurement	Scenario analysis, portfolio design under stress
Geopolitical Insight	Limited	“What just happened?” (rearview mirror)	“What might happen next?” (forward-looking)

Together, realized betas and nonparametric betas give investors both a seismograph (short-term shock detection) and a compass (longer-term, nonlinear modeling).

Realized Betas: The Seismograph of Geopolitical Shocks

When geopolitics strikes, realized betas reveal how market sensitivities shift in real time. They are derived from realized covariance and variance, often using daily or intraday data.

- **Ukraine War (2022):** Realized betas of European energy stocks against oil spiked dramatically in the first days of the invasion, reflecting immediate supply fears.
- **Brexit Referendum (2016):** The pound's realized beta versus global equities jumped, exposing how a political vote reverberated far beyond currency markets.
- **U.S.–China Trade War (2018–19):** Realized betas of semiconductor stocks surged relative to market indices, showing the sector's acute vulnerability.

These episodes highlight that betas are not constant they are volatile themselves. By diagnosing time-varying co-movements, realized betas provide investors and regulators with a real-time stress monitor.

Nonparametric Betas: The Compass in a Nonlinear World

Realized betas tell us what just happened. Nonparametric betas tell us how assets might behave under different regimes. By avoiding rigid assumptions, they capture nonlinear, state-dependent relationships.

Consider energy equities:

- In normal times, their beta relative to oil may be modest.
- During sanctions or war, the beta can surge nonlinearly, amplifying contagion.
- When risks subside, the beta may decline or even invert.

Nonparametric methods uncover these nonlinearities. They show that the relationship between assets and benchmarks is not just unstable but conditional — it depends on the political regime, the type of shock, and the level of uncertainty.

This insight is invaluable for portfolio design. Investors can identify assets with positive geopolitical betas (defense contractors, gold, energy, cybersecurity) and those with negative geopolitical betas (airlines, tourism, consumer discretionary). By building allocations that adapt to shifting regimes, they can hedge not only market risk but also political risk.

Regional and Sectoral Asymmetries

Geopolitical shocks do not spread evenly across the globe. Nonparametric and realized betas reveal regional vulnerabilities that parametric models miss:

Region	Geopolitical Shock Impact	Beta Behavior
MENA	Oil supply disruptions, conflict spillovers	Energy & FX betas spike sharply, contagion to global oil markets
Asia-Pacific	U.S.–China rivalry, supply chain shocks	Semiconductor and tech betas show nonlinear surges
Europe	Energy dependency, sanctions	Equity and currency betas highly state-dependent
U.S.	Safe-haven flows, defense spending	Defense sector shows positive geopolitical betas

Sectorally, the divide is equally stark:

- **Positive geopolitical betas:** defense, energy, gold, cybersecurity.
- **Negative geopolitical betas:** airlines, consumer goods, global tourism.

Nonparametric analysis shows these betas are not linear but asymmetric — some sectors plunge during crises yet rebound strongly afterward.

Sentiment and Behavioral Feedbacks

Geopolitical shocks are not only about fundamentals. They are amplified by investor sentiment and behavioral biases. Fear, herding, and flight-to-safety flows create sharp spikes in realized betas and nonlinear shifts in nonparametric measures. During the Arab Spring, for example, markets priced contagion far beyond directly affected countries, illustrating how perception and psychology magnify shocks.

Machine Learning and Adaptive Risk Models

Nonparametric betas align naturally with machine learning techniques. Kernel regressions, local polynomials, and random forests are already being used to estimate state-dependent betas under geopolitical stress. These methods move beyond linear models and allow for adaptive, data-driven mapping of risk exposures. As AI expands into financial analytics, the integration of machine learning with nonparametric beta estimation could redefine how we measure and price political risk. A long forgotten technique could also be applied which is GMDH Group Method of Data Handling.

Liquidity and Market Microstructure

Geopolitical shocks also manifest through liquidity channels. Bid–ask spreads widen, depth evaporates, and correlations spike across assets. Realized betas are sensitive to

these microstructure shifts, while nonparametric models capture the nonlinear contagion paths that follow. The flash crashes and sudden illiquidity during sanctions announcements or missile strikes underscore that political risk is not just about returns — it is about market functioning itself.

Commodities, FX, and Risk Premia

Geopolitical risk is most visible in commodities and FX markets. Oil, wheat, and rare-earth minerals often display nonlinear beta patterns, while safe-haven currencies (USD, CHF, JPY) exhibit negative or regime-dependent betas.

Investors increasingly demand geopolitical risk premia — higher expected returns for bearing exposure to unstable regions or sectors. Nonparametric betas can isolate these premia, revealing which assets systematically price political uncertainty.

Entropy and Historical Continuity

Geopolitical shocks increase systemic entropy — disorder in capital markets. Traditional betas cannot capture the jump in complexity; realized betas show the spikes, while nonparametric betas map the nonlinear feedback loops.

This is not new. During the oil shocks of the 1970s and the Cold War, markets behaved nonlinearly, but the models of the time were too blunt to capture it. Today, with better data and more flexible methods, we can finally see what was once hidden: the true complexity of political-financial interactions.

Policy and Strategic Implications

- **For central banks:** Realized betas provide near real-time measures of systemic stress.
- **For sovereign wealth funds & pensions:** Nonparametric betas allow adaptive hedging of geopolitical tail risks.
- **For multilateral institutions (IMF, BIS):** These tools model nonlinear contagion channels across borders, improving crisis-preparedness.
- **For regulators:** Monitoring liquidity stress and market microstructure shifts through beta dynamics can guide circuit breakers and stabilization policies.

The broader message is that political risk is now financial risk. Treating it as exogenous noise is no longer viable.

Operational Guide: Beta Seismograph and Compass:

1. The Beta Seismograph – A Risk Alarm System

Purpose: Detect sudden changes in how an asset reacts to the broader market — the financial equivalent of spotting tremors before an earthquake.

Steps to construct and use:

1. **Collect high-frequency data:** Track prices of the asset and the benchmark index in short intervals (like 5-minute or hourly moves).
2. **Measure co-movement:** Continuously calculate how tightly the asset is moving with the market.
3. **Build a rolling history:** Compare the latest readings with the typical range from the past few days or weeks.
4. **Set thresholds:** Define what counts as “normal” vs. “abnormal” movement.
5. **Trigger alerts:** If the relationship jumps suddenly — either sharply higher or lower — flag it as a potential “beta quake.”
6. **Cross-check with liquidity:** Confirm whether this spike coincides with lower trading depth, higher spreads, or unusual volumes, which makes the tremor more serious.

Use case: A defense stock suddenly doubles its sensitivity to the market after war headlines. The seismograph raises an alarm, allowing risk managers to hedge quickly.

2. The Beta Compass – A Scenario Mapping Tool

Purpose: Show how an asset’s risk sensitivity changes under different political and market climates — a navigational chart for uncertainty.

Steps to construct and use:

1. **Choose key states of the world:** Examples include oil prices, volatility levels, interest rates, or a geopolitical risk index.
2. **Link past experience to conditions:** For each combination of these states, record how the assets responded to the market historically.
3. **Draw a flexible map:** Instead of forcing straight-line patterns, let the data reveal complex shapes and curves in the relationship.

4. **Build a compass surface:** This map shows how the asset's beta changes as conditions shift — calm seas, rising tensions, or outright crisis.
5. **Run scenarios:** Ask “what if” questions — What happens to beta if oil surges 20% and volatility doubles? Does the asset become safer, or more exposed?
6. **Guide strategy:** Use the compass to stress-test portfolios, plan hedges, and prepare for policy decisions under different geopolitical paths.

Use case: A central bank tests how its domestic banks' betas respond if global volatility spikes and oil prices climb. The compass shows vulnerability in one state, resilience in another — guiding contingency planning.

3. Using Both Tools Together

- **Seismograph = Short-term warning system.** Alerts when beta shifts suddenly in real time.
- **Compass = Strategic navigation system.** Provides a broader view of how beta evolves under different geopolitical and market conditions.
- **Fusion:** When the seismograph alarm rings, check the compass to see where you are in the risk map — and where the needle might move next.

Conclusion

Geopolitical risk has become the defining driver of global markets, not a peripheral disturbance. Traditional models rooted in parametric beta and the assumptions of stability, linearity, and Gaussian distribution are no longer sufficient to capture the volatility, asymmetry, and contagion unleashed by political shocks. By framing realized betas as a seismograph and nonparametric betas as a compass, this paper introduces a dual framework that both monitors immediate tremors and guides strategic navigation. Together, they provide investors, policymakers, and regulators with tools capable of addressing the complexity of a multipolar and politically unstable financial order.

The evidence is clear: realized betas allow for real-time detection of market stress, while nonparametric betas model nonlinear, regime-dependent sensitivities. This approach uncovers regional and sectoral asymmetries, identifies positive and negative geopolitical exposures, and integrates sentiment, liquidity, and microstructure into risk assessment. Moreover, the Python-based demonstration proves that these concepts are not merely theoretical — they can be operationalized into functional systems that issue alerts, simulate crises, and map scenario surfaces. In doing so, the paper bridges theory and practice, equipping stakeholders to anticipate rather than merely react to shocks.

Policy Recommendations

1. Central Banks: Incorporate realized betas into systemic risk dashboards to monitor geopolitical stress in near real-time. Alerts should feed into liquidity provision frameworks, circuit breaker designs, and macroprudential policy.
2. Sovereign Wealth Funds and Pensions: Use nonparametric beta analysis to diversify portfolios dynamically, hedging against geopolitical tail risks while capturing risk premia in sectors with positive exposure (e.g., defense, energy, cybersecurity).
3. Multilateral Institutions (IMF, BIS, World Bank): Develop cross-border contagion maps using beta dynamics, enabling coordinated responses to shocks that spread through trade, commodities, and capital flows.
4. Regulators and Market Supervisors: Monitor liquidity stress through realized beta instability and integrate these signals into early-warning systems for financial stability. Tailor regulations to address nonlinear spillovers rather than assuming linear responses.
5. Investors and Risk Managers: Treat political risk as endogenous to financial markets, not as exogenous noise. Deploy seismograph-and-compass systems to integrate geopolitical variables into portfolio design, stress testing, and scenario analysis.

Tips for Practical Implementation:

- Data Infrastructure: Build pipelines to collect high-frequency asset, commodity, and FX data, alongside geopolitical indicators such as risk indices, sanctions announcements, and conflict metrics.
- Automation: Implement real-time monitoring systems like the Python-based demo, ensuring automated alerts for beta spikes, crashes, or regime shifts.
- Scenario Libraries: Develop stress scenarios based on historical and simulated geopolitical shocks (oil embargoes, cyberattacks, sanctions regimes) and test portfolio resilience under these conditions.
- Integration with AI/ML: Enhance nonparametric beta estimation with machine learning models (random forests, neural networks) to capture nonlinearities and adapt as new data arrives.
- Governance and Training: Equip financial institutions and policymakers with both technical capacity and decision protocols to interpret beta alerts and translate them into actionable strategies.

- **Continuous Calibration:** Treat the seismograph and compass as living systems — update models regularly as markets evolve, shocks recur, and political dynamics shift.

In sum, the fusion of realized and nonparametric betas creates a framework that is both diagnostic and predictive. It enables stakeholders to detect tremors before they escalate into crises and to chart courses through uncharted geopolitical waters. The policy challenge is no longer whether to account for political risk, but how. The answer lies in operationalizing adaptive, data-driven systems that make geopolitical risk visible, measurable, and, above all, manageable.

Limitations

Although this paper advances a novel conceptual framework that positions realized and nonparametric betas as critical tools for understanding geopolitical risk, several limitations must be acknowledged.

First, the contribution remains conceptual rather than empirical. The framework is illustrated with reference to high-profile shocks such as Brexit, the U.S.–China trade war, and the Ukraine invasion, but it does not yet provide a systematic, data-driven validation across markets, regions, or political regimes. As a result, the conclusions should be viewed as hypotheses that require empirical testing before they can be generalized.

Second, the framework is constrained by data availability and quality. Realized betas require high-frequency market data, while nonparametric betas depend on state variables that may not be consistently available across all geographies. Emerging markets — often at the epicenter of geopolitical risk — tend to lack the necessary data infrastructure, creating a structural bias toward advanced economies and highly liquid asset classes. This limitation raises questions about the global applicability of the framework.

Third, reliance on adaptive methods such as machine learning introduces risks of overfitting and reduced interpretability. While these methods excel at capturing nonlinearities, they can produce opaque results that complicate communication to policymakers and regulators, who often require transparency in risk metrics. The challenge lies in balancing predictive accuracy with interpretability.

Fourth, the framework primarily captures short- and medium-term market dynamics, but many geopolitical shocks also produce long-term structural effects — regulatory reforms, capital controls, trade realignments — that fall outside the immediate scope of beta estimation. Behavioral amplification, such as herding or contagion driven by investor sentiment, is acknowledged but remains difficult to quantify systematically.

Finally, practical implementation presents challenges. While the Python-based demonstration shows feasibility, scaling such systems to institutional settings requires robust data pipelines, governance protocols, and training. Institutions with limited technological capacity may struggle to operationalize these tools, potentially widening the gap between advanced and developing markets.

Future Research Directions

The limitations of this study point directly toward a rich agenda for future research. Four areas in particular stand out.

1. Empirical Validation Across Markets

Future work should undertake large-scale empirical testing of realized and nonparametric betas across asset classes (equities, commodities, FX, bonds), geographies (developed and emerging markets), and time horizons (intraday to multi-year). Event studies of geopolitical shocks — elections, sanctions, cyberattacks, conflicts — could test whether the framework consistently identifies regime-dependent sensitivities.

2. Methodological Refinement

Methodological innovation will be essential. Hybrid approaches that combine econometric structure with machine learning flexibility could improve both accuracy and interpretability. Bayesian nonparametric methods, deep learning architectures, and advanced estimators such as Hayashi–Yoshida covariance and jump-detection techniques should be explored to improve robustness under volatile conditions.

3. Integration with Political Variables

Future studies should embed political economy variables directly into beta estimation. Indicators of regime stability, sanctions intensity, geopolitical risk indices, or even sentiment measures derived from natural language processing of news flows could enrich explanatory power. Cross-disciplinary collaboration with political science and international relations will be crucial in advancing this integration.

4. Policy Application and Stress Testing

Finally, the framework should be tested in policy and institutional contexts. Beta-based measures could be embedded into macroprudential stress testing frameworks for central banks, multilateral institutions, and regulators. Case studies with sovereign wealth funds, pensions, and large asset managers could assess how adaptive beta modeling improves hedging, allocation, and the pricing of geopolitical risk premia.

Taken together, these research avenues would move the framework from conceptual proposal to operational practice. By expanding empirical evidence, refining methods, incorporating political variables, and embedding applications in policy, the seismograph-and-compass approach could evolve into a robust architecture for navigating financial risk in a multipolar and geopolitically unstable world.

References

Viktor Todorov and Tim Bollerslev, **“Jumps and Betas: A New Framework for Disentangling and Estimating Systematic Risks”**, 2007, *CREATES Research Paper No. 2007-15*. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=1150066

Nobel Laureate Robert Engle, **“Measuring and Hedging Geopolitical Risk”**, 2021, Youtube Video, <https://www.youtube.com/watch?v=kDYJtcbCV4Q&t=18s>

Vincent James, Kevin Ng and Jonathan J. Reeves, **“Monthly Forecasts of Systematic Risk: An Evaluation”**, 2007.
<https://ssrn.com/abstract=1031551> or <http://dx.doi.org/10.2139/ssrn.1031551>

Vincent J. Hooper, Kevin Ng, Jonathan J. Reeves, **“Quarterly beta forecasting: An evaluation”**, *International Journal of Forecasting*, Volume 24, Issue 3, 2008, Pages 480-489. <https://www.sciencedirect.com/science/article/abs/pii/S016920700800037X>

Python Code:

Key Components:

1. BetaSeismograph Class - The "Earthquake Detector"

Calculates realized betas using rolling covariance/variance

Monitors beta instability in real-time

Generates alerts when beta relationships spike or crash

Tracks liquidity conditions (volume, spreads)

2. BetaCompass Class - The "Scenario Navigator"

Uses machine learning (Random Forest) for nonparametric beta modeling

Maps how betas change under different geopolitical conditions

Creates scenario surfaces showing beta sensitivity

Provides feature importance analysis

3. GeopoliticalRiskAnalyzer Class - The Complete System

Combines both seismograph and compass

Handles real-time data streaming

Provides scenario analysis capabilities

Generates visualizations

Key Features

Real-time Monitoring: The seismograph continuously tracks beta changes and alerts when relationships shift dramatically (like during the Ukraine invasion or Brexit).

Scenario Modeling: The compass predicts how assets will behave under different conditions (oil shocks, market panics, etc.).

Nonlinear Analysis: Captures the complex, state-dependent relationships that traditional linear models miss.

Practical Implementation: Ready-to-use system with data ingestion, model training, and visualization capabilities. The demo function shows how the system works with simulated geopolitical events, demonstrating both the seismograph's real-time alerts and the compass's scenario analysis. This implementation transforms the theoretical framework from the document into actionable risk management tools for investors and policymakers.

Sample Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import warnings
warnings.filterwarnings('ignore')

class BetaSeismograph:
    """
    Real-time beta shock detection system - Financial earthquake detector
    """
    def __init__(self, lookback_period=20, alert_threshold=2.0):
        self.lookback_period = lookback_period
        self.alert_threshold = alert_threshold
        self.beta_history = []
        self.volatility_history = []
        self.alerts = []

    def calculate_realized_beta(self, asset_returns, market_returns, window=5):
        """Calculate realized beta using rolling covariance/variance"""
        if len(asset_returns) < window or len(market_returns) < window:
            return np.nan
```

```

asset_series = pd.Series(asset_returns)
market_series = pd.Series(market_returns)

# Rolling realized covariance and variance
rolling_cov = asset_series.rolling(window).cov(market_series).iloc[-1]
rolling_var = market_series.rolling(window).var().iloc[-1]

if rolling_var == 0:
    return np.nan

return rolling_cov / rolling_var

def update_seismograph(self, asset_returns, market_returns, timestamp,
                        volume=None, spread=None):
    """Update the seismograph with new data and check for alerts"""

    # Calculate current realized beta
    current_beta = self.calculate_realized_beta(asset_returns,
market_returns)

    if np.isnan(current_beta):
        return None

    self.beta_history.append({
        'timestamp': timestamp,
        'beta': current_beta,
        'volume': volume,
        'spread': spread
    })

    # Calculate volatility of beta (beta instability)
    if len(self.beta_history) >= self.lookback_period:
        recent_betas = [x['beta'] for x in self.beta_history[-
self.lookback_period:]]
        beta_vol = np.std(recent_betas)
        beta_mean = np.mean(recent_betas)

        # Z-score of current beta
        if beta_vol > 0:
            z_score = (current_beta - beta_mean) / beta_vol

            # Check for alert condition
            if abs(z_score) > self.alert_threshold:
                alert = {

```

```

        'timestamp': timestamp,
        'beta': current_beta,
        'z_score': z_score,
        'type': 'SPIKE' if z_score > 0 else 'CRASH',
        'severity': 'HIGH' if abs(z_score) > 3 else 'MEDIUM',
        'volume': volume,
        'spread': spread
    }
    self.alerts.append(alert)
    return alert

    return None

def get_current_status(self):
    """Get current seismograph reading"""
    if not self.beta_history:
        return "No data"

    current = self.beta_history[-1]
    if len(self.beta_history) >= self.lookback_period:
        recent_betas = [x['beta'] for x in self.beta_history[-self.lookback_period:]]
        beta_vol = np.std(recent_betas)
        status = "VOLATILE" if beta_vol > 0.5 else "STABLE"
    else:
        status = "CALIBRATING"

    return f"Beta: {current['beta']:.3f} | Status: {status}"

class BetaCompass:
    """
    Scenario mapping tool for nonparametric beta analysis
    """
    def __init__(self):
        self.model = RandomForestRegressor(n_estimators=100, random_state=42)
        self.scaler = StandardScaler()
        self.is_fitted = False
        self.feature_names = []

    def prepare_data(self, asset_returns, market_returns, state_variables):
        """
        Prepare data for nonparametric beta modeling

        Parameters:
        asset_returns: array of asset returns

```

```

market_returns: array of market returns
state_variables: dict of state variables (oil_price, vix, rates, etc.)
"""

# Calculate rolling betas as target variable
betas = []
features = []

window = 20
for i in range(window, len(asset_returns)):
    # Calculate beta for this window
    asset_window = asset_returns[i-window:i]
    market_window = market_returns[i-window:i]

    covariance = np.cov(asset_window, market_window)[0, 1]
    market_var = np.var(market_window)

    if market_var > 0:
        beta = covariance / market_var
        betas.append(beta)

    # Collect state variables for this period
    feature_row = []
    for var_name, var_values in state_variables.items():
        if i < len(var_values):
            feature_row.append(var_values[i])
        else:
            feature_row.append(var_values[-1]) # Use last available
value

    # Add market conditions
    feature_row.extend([
        np.std(market_window), # Market volatility
        np.mean(market_window), # Market trend
        np.std(asset_window), # Asset volatility
    ])

    features.append(feature_row)

# Set feature names
self.feature_names = list(state_variables.keys()) + [
    'market_vol', 'market_trend', 'asset_vol'
]

return np.array(features), np.array(betas)

```

```

def fit_compass(self, asset_returns, market_returns, state_variables):
    """Fit the nonparametric beta model"""

    X, y = self.prepare_data(asset_returns, market_returns, state_variables)

    if len(X) == 0:
        raise ValueError("Not enough data to fit model")

    # Scale features
    X_scaled = self.scaler.fit_transform(X)

    # Fit model
    self.model.fit(X_scaled, y)
    self.is_fitted = True

    return self

def predict_beta(self, state_scenario):
    """
    Predict beta under specific scenario conditions

    Parameters:
    state_scenario: dict with same keys as used in fitting
    """
    if not self.is_fitted:
        raise ValueError("Model must be fitted before prediction")

    # Create feature vector from scenario
    feature_vector = []
    for feature_name in self.feature_names:
        if feature_name in state_scenario:
            feature_vector.append(state_scenario[feature_name])
        else:
            feature_vector.append(0) # Default value

    # Scale and predict
    X_scaled = self.scaler.transform([feature_vector])
    predicted_beta = self.model.predict(X_scaled)[0]

    return predicted_beta

def create_scenario_surface(self, var1_name, var1_range, var2_name,
var2_range,
                           base_scenario=None):

```

```

        """Create a 2D surface showing how beta changes with two variables"""

        if not self.is_fitted:
            raise ValueError("Model must be fitted before creating scenarios")

        if base_scenario is None:
            base_scenario = {name: 0 for name in self.feature_names}

        # Create grid
        var1_values = np.linspace(var1_range[0], var1_range[1], 20)
        var2_values = np.linspace(var2_range[0], var2_range[1], 20)

        beta_surface = np.zeros((len(var2_values), len(var1_values)))

        for i, var2_val in enumerate(var2_values):
            for j, var1_val in enumerate(var1_values):
                scenario = base_scenario.copy()
                scenario[var1_name] = var1_val
                scenario[var2_name] = var2_val

                beta_surface[i, j] = self.predict_beta(scenario)

        return var1_values, var2_values, beta_surface

    def get_feature_importance(self):
        """Get feature importance from the fitted model"""
        if not self.is_fitted:
            return None

        importance = self.model.feature_importances_
        return dict(zip(self.feature_names, importance))

class GeopoliticalRiskAnalyzer:
    """
    Main class combining seismograph and compass for comprehensive analysis
    """
    def __init__(self):
        self.seismograph = BetaSeismograph()
        self.compass = BetaCompass()
        self.data_history = []

    def add_market_data(self, timestamp, asset_price, market_price,
                       state_variables=None, volume=None, spread=None):
        """Add new market data point"""

```

```

        self.data_history.append({
            'timestamp': timestamp,
            'asset_price': asset_price,
            'market_price': market_price,
            'state_variables': state_variables or {},
            'volume': volume,
            'spread': spread
        })

    # Calculate returns if we have enough data
    if len(self.data_history) >= 2:
        asset_returns = [
            (self.data_history[i]['asset_price'] / self.data_history[i-1]['asset_price'] - 1)
            for i in range(1, len(self.data_history))
        ]
        market_returns = [
            (self.data_history[i]['market_price'] / self.data_history[i-1]['market_price'] - 1)
            for i in range(1, len(self.data_history))
        ]

        # Update seismograph
        alert = self.seismograph.update_seismograph(
            asset_returns, market_returns, timestamp, volume, spread
        )

        return alert

    return None

def train_compass(self):
    """Train the compass using historical data"""

    if len(self.data_history) < 50:
        raise ValueError("Need at least 50 data points to train compass")

    # Extract returns and state variables
    asset_returns = []
    market_returns = []
    state_vars = {}

    for i in range(1, len(self.data_history)):
        asset_ret = (self.data_history[i]['asset_price'] /
                     self.data_history[i-1]['asset_price'] - 1)

```

```

        market_ret = (self.data_history[i]['market_price'] /
                       self.data_history[i-1]['market_price'] - 1)

        asset_returns.append(asset_ret)
        market_returns.append(market_ret)

        # Collect state variables
        for var_name, var_value in
self.data_history[i]['state_variables'].items():
            if var_name not in state_vars:
                state_vars[var_name] = []
            state_vars[var_name].append(var_value)

        # Train compass
        self.compass.fit_compass(asset_returns, market_returns, state_vars)

        return True

def scenario_analysis(self, scenarios):
    """Run multiple scenario analyses"""

    results = {}
    for scenario_name, scenario_values in scenarios.items():
        try:
            beta = self.compass.predict_beta(scenario_values)
            results[scenario_name] = beta
        except Exception as e:
            results[scenario_name] = f"Error: {str(e)}"

    return results

def plot_seismograph(self, last_n=100):
    """Plot recent seismograph readings"""

    if len(self.seismograph.beta_history) == 0:
        print("No seismograph data to plot")
        return

    history = self.seismograph.beta_history[-last_n:]
    timestamps = [x['timestamp'] for x in history]
    betas = [x['beta'] for x in history]

    plt.figure(figsize=(12, 6))
    plt.plot(timestamps, betas, 'b-', linewidth=1, alpha=0.7)

```

```

        plt.axhline(y=np.mean(betas), color='g', linestyle='--', alpha=0.5,
label='Mean')

    # Mark alerts
    for alert in self.seismograph.alerts:
        if alert['timestamp'] in timestamps:
            color = 'red' if alert['type'] == 'SPIKE' else 'orange'
            plt.axvline(x=alert['timestamp'], color=color, linestyle='-',
alpha=0.8)

    plt.title('Beta Seismograph - Real-time Risk Monitor')
    plt.xlabel('Time')
    plt.ylabel('Realized Beta')
    plt.xticks(rotation=45)
    plt.legend()
    plt.tight_layout()
    plt.show()

def plot_compass_surface(self, var1_name, var1_range, var2_name, var2_range):
    """Plot 2D scenario surface from compass"""

    if not self.compass.is_fitted:
        print("Compass must be trained before plotting")
        return

    try:
        var1_vals, var2_vals, surface = self.compass.create_scenario_surface(
            var1_name, var1_range, var2_name, var2_range
        )

        plt.figure(figsize=(10, 8))
        contour = plt.contourf(var1_vals, var2_vals, surface, levels=20,
cmap='RdYlBu_r')
        plt.colorbar(contour, label='Predicted Beta')
        plt.xlabel(var1_name)
        plt.ylabel(var2_name)
        plt.title('Beta Compass - Scenario Surface')
        plt.show()

    except Exception as e:
        print(f"Error creating surface: {str(e)}")

# Example usage and demonstration
def demo_geopolitical_risk_system():
    """Demonstrate the geopolitical risk analysis system"""

```

```

print("=== Geopolitical Risk Analysis System Demo ===\n")

# Initialize analyzer
analyzer = GeopoliticalRiskAnalyzer()

# Simulate market data with geopolitical events
np.random.seed(42)
n_days = 200

print("1. Generating synthetic market data...")

# Base market conditions
market_prices = [100]
asset_prices = [95]

# State variables (oil price, VIX, geopolitical risk index)
oil_prices = [60]
vix_levels = [20]
geo_risk = [0]

# Simulate geopolitical events
crisis_periods = [50, 120, 180] # Days when crises occur

for day in range(1, n_days):
    # Market dynamics
    market_shock = np.random.normal(0, 0.02)
    asset_shock = np.random.normal(0, 0.025)

    # Geopolitical crisis effects
    if day in crisis_periods:
        geo_risk_shock = np.random.uniform(0.5, 1.0) # Crisis starts
        oil_shock = np.random.uniform(0.1, 0.3) # Oil spikes
        vix_shock = np.random.uniform(0.3, 0.6) # Fear spikes
    elif any(abs(day - crisis) < 10 for crisis in crisis_periods):
        geo_risk_shock = np.random.uniform(-0.1, 0.1) # Crisis continues/fades
        oil_shock = np.random.normal(0, 0.05)
        vix_shock = np.random.normal(0, 0.1)
    else:
        geo_risk_shock = np.random.normal(0, 0.05)
        oil_shock = np.random.normal(0, 0.03)
        vix_shock = np.random.normal(0, 0.08)

    # Update state variables
    current_geo_risk = max(0, geo_risk[-1] + geo_risk_shock)

```

```

current_oil = max(30, oil_prices[-1] * (1 + oil_shock))
current_vix = max(10, min(80, vix_levels[-1] * (1 + vix_shock)))

geo_risk.append(current_geo_risk)
oil_prices.append(current_oil)
vix_levels.append(current_vix)

# Asset responds to market + geopolitical factors
geo_beta = 0.5 + current_geo_risk # Beta increases with geopolitical risk
asset_return = geo_beta * market_shock + asset_shock + 0.1 * geo_risk_shock

market_prices.append(market_prices[-1] * (1 + market_shock))
asset_prices.append(asset_prices[-1] * (1 + asset_return))

# Add to analyzer
state_vars = {
    'oil_price': current_oil,
    'vix': current_vix,
    'geo_risk': current_geo_risk
}

alert = analyzer.add_market_data(
    timestamp=day,
    asset_price=asset_prices[-1],
    market_price=market_prices[-1],
    state_variables=state_vars,
    volume=np.random.uniform(1000, 5000),
    spread=np.random.uniform(0.001, 0.01)
)

if alert:
    print(f"🚨 SEISMOGRAPH ALERT on day {day}: {alert['type']} "
          f"(Beta: {alert['beta']:.3f}, Z-score: {alert['z_score']:.2f})")

    print(f"\n2. Seismograph detected {len(analyzer.seismograph.alerts)} beta
shocks")
    print(f"    Current status: {analyzer.seismograph.get_current_status()}")

# Train compass
print("\n3. Training Beta Compass...")
analyzer.train_compass()

# Show feature importance
importance = analyzer.compass.get_feature_importance()
print("\n    Feature Importance in Beta Prediction:")

```

```

        for feature, imp in sorted(importance.items(), key=lambda x: x[1],
reverse=True):
            print(f"    {feature}: {imp:.3f}")

# Scenario analysis
print("\n4. Running Scenario Analysis...")

scenarios = {
    'Peaceful Times': {'oil_price': 65, 'vix': 15, 'geo_risk': 0.1},
    'Oil Crisis': {'oil_price': 120, 'vix': 35, 'geo_risk': 0.8},
    'Market Panic': {'oil_price': 80, 'vix': 60, 'geo_risk': 1.2},
    'War Scenario': {'oil_price': 150, 'vix': 45, 'geo_risk': 2.0},
    'Recovery Phase': {'oil_price': 70, 'vix': 25, 'geo_risk': 0.3}
}

results = analyzer.scenario_analysis(scenarios)

print("    Predicted Betas under different scenarios:")
for scenario, beta in results.items():
    if isinstance(beta, float):
        risk_level = "HIGH" if beta > 1.5 else "MEDIUM" if beta > 1.0 else
"LOW"
        print(f"    {scenario}: {beta:.3f} ({risk_level} risk)")
    else:
        print(f"    {scenario}: {beta}")

# Plot results
print("\n5. Generating visualizations...")
analyzer.plot_seismograph()
analyzer.plot_compass_surface('oil_price', (50, 150), 'vix', (10, 60))

print("\n=== Analysis Complete ===")
print("\nKey Insights:")
print("• The Seismograph provides real-time alerts when beta relationships
shift suddenly")
print("• The Compass reveals how betas change nonlinearly under different
conditions")
print("• Together, they offer both immediate warning and strategic scenario
planning")
print("• This approach moves beyond traditional linear models to capture
geopolitical complexity")

if __name__ == "__main__":
    demo_geopolitical_risk_system()

```

Synthesis: The Seismograph and Compass Framework

The following table summarizes the core components, functions, and applications of the proposed dual-framework for adaptive geopolitical risk analysis.

Aspect	Realized Beta (The Seismograph)	Nonparametric Beta (The Compass)	Combined Framework
Primary Function	Detection & Alerting	Navigation & Scenario Planning	Integrated Risk Management
Time Horizon	Short-Term (Real-Time)	Medium to Long-Term (Forward-Looking)	Multi-Horizon Analysis
Core Methodology	High-Frequency Covariance/Variance	Machine Learning (e.g., Random Forest)	Hybrid Data-Driven Approach
Key Output	Real-time beta value and instability Z-score	Predicted beta under a given scenario	Alerts + Scenario Mappings
Answers the Question	"What is happening right now?"	"What could happen if...?"	"What is happening now, and what might come next?"
Ideal For	Monitoring stress events, liquidity shocks	Portfolio design, hedging strategy, stress testing	Strategic decision-making under uncertainty

Aspect	Realized Beta (The Seismograph)	Nonparametric Beta (The Compass)	Combined Framework
Example Use Case	Alerting a 200% spike in a defense stock's beta within hours of a conflict escalation.	Modeling how an energy portfolio's risk changes if oil hits \$150 and volatility doubles.	Receiving a seismograph alert and using the compass to simulate the next week's scenarios.
Primary Users	Traders, Risk Managers, Central Banks	Portfolio Managers, Strategists, SWFs	Chief Investment Officers, Policymakers, Regulators
Required Data	High-Frequency Price Data (Asset & Benchmark)	State Variables (e.g., VIX, Oil Price, Risk Indices)	Combined market data and geopolitical indicators